Zephyr RTOS: архитектура и устройство

2026

Содержание

1. Зачем нужны RTOS?
2. Отличия Zephyr от других RTOS
3. Архитектура Zephyr
4. Ядро Zephyr
5. Потоки
6. Планировщик
7. Управление памятью
8. Заключение

От суперцикла к RTOS

Проблема суперцикла

- Один `while(1)` обрабатывает всё
- С ростом функциональности — хаос
- Разные задачи требуют разного темпа
- Блокирующая операция (UART, TCP/IP) ломает тайминги

Эволюция

1. Суперцикл + временные метки
2. Кооперативный планировщик
3. Вытесняющий планировщик (RTOS)

Вывод

RTOS нужна, когда сложность превышает возможности одного цикла: TCP/IP + мотор + роборука + дисплей одновременно.

Кооперативный vs вытесняющий планировщик

Кооперативный

- Задача сама отдаёт управление
- Одна медленная задача блокирует все остальные
- Прост в реализации, но не масштабируется

Кооперативный vs вытесняющий планировщик

Кооперативный

- Задача сама отдаёт управление
- Одна медленная задача блокирует все остальные
- Прост в реализации, но не масштабируется

Вытесняющий (preemptive) — основа RTOS

- Аппаратный таймер (например, 1 мс) вызывает прерывание
- Планировщик может **принудительно** переключить задачу
- Сохраняется/восстанавливается контекст (регистры, стек)
- Медленная задача не мешает критичным по времени

Zephyr: что выделяет его среди RTOS

1. **Linux Foundation** — поддержка крупнейшего open-source фонда
2. **Родство с Linux** — Kconfig, DeviceTree, общие концепции
3. **Гибкая конфигурация** — Kconfig позволяет включить только нужное
4. **Широкая поддержка архитектур**

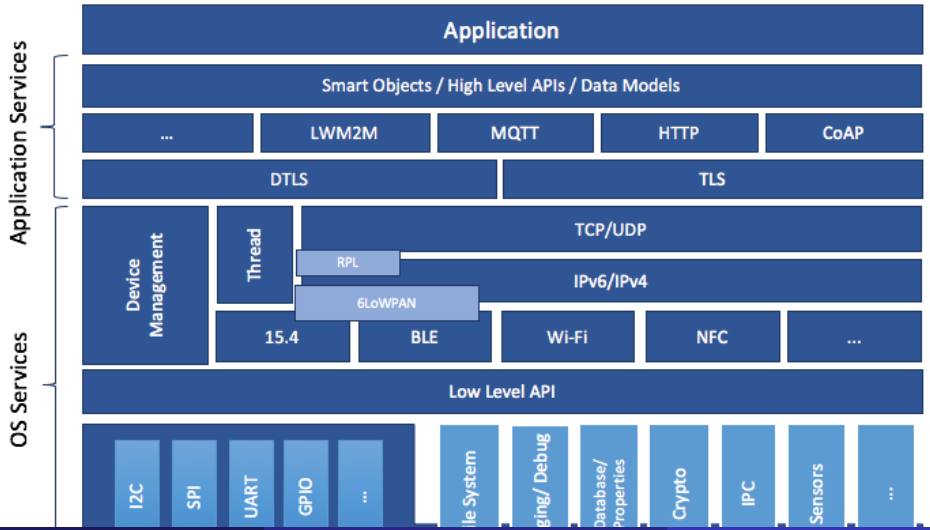
Zephyr: что выделяет его среди RTOS

1. **Linux Foundation** — поддержка крупнейшего open-source фонда
2. **Родство с Linux** — Kconfig, DeviceTree, общие концепции
3. **Гибкая конфигурация** — Kconfig позволяет включить только нужное
4. **Широкая поддержка архитектур**

Поддерживаемые архитектуры

ARCv2/ARCv3 | ARMv6-M/v7-M/v8-M (Cortex-M) | ARMv7-A/v8-A (Cortex-A, 32/64-bit) | ARMv7-R/v8-R (Cortex-R) | Intel x86 (32/64-bit) | MIPS32 | OpenRISC | Renesas RX | **RISC-V (32/64-bit)** | SPARC V8 | Tensilica Xtensa

Системная архитектура Zephyr



Структура проекта Zephyr

Ключевые файлы

- `CMakeLists.txt` — сборка
- `prj.conf` — конфигурация (Kconfig)
- `src/` — исходный код
- `build/` — артефакты сборки

Пример prj.conf

```
CONFIG_PRINTK=y
CONFIG_SERIAL=y
CONFIG_HEAP_MEM_POOL_SIZE=256
```

Ядро Zephyr: основа каждого приложения

Характеристики

- Малозатратная многопоточная среда
- Минимальный footprint — **от 2 КБ**
- Для сложных систем — 50–900 КБ (BLE, Wi-Fi, сложная многопоточность)
- Модульная архитектура: включается только нужное

Сервисы ядра

Планировщик | Управление потоками | Синхронизация (семафоры, мьютексы, очереди) | Управление памятью | Таймеры | Обработка прерываний

Поток (Thread) — что это?

Определение

Поток — изолированный экземпляр, выполняющий некоторую задачу. RTOS создаёт иллюзию одновременного выполнения на одном CPU путём переключения контекста.

Поток (Thread) — что это?

Определение

Поток — изолированный экземпляр, выполняющий некоторую задачу. RTOS создаёт иллюзию одновременного выполнения на одном CPU путём переключения контекста.

Составные части потока

Стек область памяти под стек потока. `#define STACKSIZE 1024`

Блок управления `struct k_thread` — метаданные потока

Точка входа функция, вызываемая при запуске

Приоритет политика планирования

Режим супервизор (по умолчанию) или пользователь

Пример создания потока

```
#include <zephyr/kernel.h>

#define STACKSIZE 1024
K_THREAD_STACK_DEFINE(thread_a_stack, STACKSIZE);
static struct k_thread thread_a_data;

void thread_a_entry(void *d1, void *d2, void *d3)
{
    ARG_UNUSED(d1); ARG_UNUSED(d2); ARG_UNUSED(d3);
    hello_loop(__func__, &thread_a_sem, &thread_b_sem);
}

void main(void)
{
    k_thread_create(&thread_a_data, thread_a_stack,
        K_THREAD_STACK_SIZEOF(thread_a_stack),
        thread_a_entry, NULL, NULL, NULL,
```

Планировщик (Scheduler)

Принцип работы

Приоритетный планировщик: потоки разделяют процессорное время.
Переключение происходит в **точках изменения расписания**:

- `k_sem_take()` / `k_sleep()` — поток засыпает
- `k_sem_give()` / `k_thread_start()` — пробуждение
- Возврат из ISR
- `k_yield()` — явная передача управления

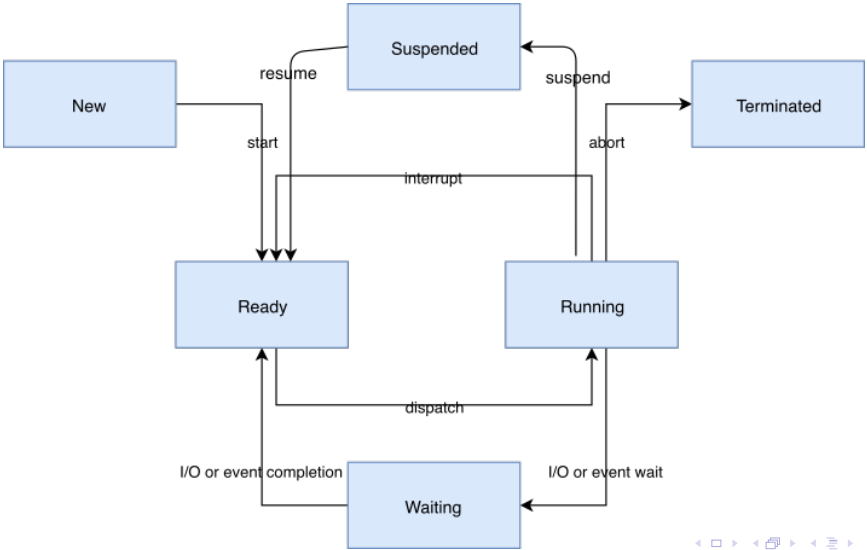
Сохранение контекста

При переключении ядро сохраняет:

- Регистры CPU
- Указатель стека
- Счётчик команд

При возобновлении — восстанавливает.

Состояния потоков



Тайм-слайсинг и сон

Тайм-слайсинг

- Время делится на **временные интервалы** (такты)
- В конце интервала — неявный `k_yield()` для вытесняемых потоков
- Не гарантирует равного времени, но гарантирует: поток не выполняется дольше одного интервала без необходимости

Тайм-слайсинг и сон

Тайм-слайсинг

- Время делится на **временные интервалы** (такты)
- В конце интервала — неявный `k_yield()` для вытесняемых потоков
- Не гарантирует равного времени, но гарантирует: поток не выполняется дольше одного интервала без необходимости

Управление выполнением

`k_sleep()`	отложить обработку на N мс. CPU освобождается
`k_wakeup()`	разбудить спящий поток преждевременно
`k_sched_lock()`	временно запретить вытеснение (критическая секция)
`k_cpu_idle()`	перевести процессор в режим ожидания (c_idle.c)
`k_yield()`	явно уступить CPU

Типы памяти во встраиваемых системах

SRAM

- Рабочая память (runtime)
- Минимальная задержка
- Куча, стеки, глобальные переменные
- КБ–МБ

Flash

- Энергонезависимая
- Код + константы
- Execute-in-place (XIP)
- Медленная запись

Cache

- Буфер CPU–память
- Кэш команд и данных
- Критичен для производительности

Регионы памяти Zephyr



Статическое vs динамическое выделение

Статическое

- Выделение на этапе компиляции
- Нулевые накладные расходы
- Детерминированное поведение
- Нет риска фрагментации
- Идеально для систем реального времени

Динамическое

- Гибкость во время выполнения
- Эффективное использование памяти
- Структуры переменного размера
- Риск: фрагментация, отказ выделения
- Накладные расходы runtime

```
1 // Static allocation
2 static uint8_t sensor_buffer[1024];
3 K_THREAD_STACK_DEFINE(sensor_stack, 2048);
4
5 // Dynamic allocation
6 void *buf = k_heap_alloc(&app_heap, 256, K_FOREVER);
```

API кучи Zephyr

Статическая куча

```
1 K_HEAP_DEFINE(app_heap, 8192); // at compile time
```

API кучи Zephyr

Статическая куча

```
K_HEAP_DEFINE(app_heap, 8192); // at compile time
```

Динамическая куча (runtime)

```
static uint8_t heap_mem[8192];
static struct k_heap custom_heap;
k_heap_init(&custom_heap, heap_mem, sizeof(heap_mem));
```

API кучи Zephyr

Статическая куча

```
K_HEAP_DEFINE(app_heap, 8192); // at compile time
```

Динамическая куча (runtime)

```
static uint8_t heap_mem[8192];
static struct k_heap custom_heap;
k_heap_init(&custom_heap, heap_mem, sizeof(heap_mem));
```

Функции

```
void *k_heap_alloc(struct k_heap *h, size_t bytes, k_timeout_t t);
void *k_heap_calloc(struct k_heap *h, size_t n, size_t sz, k_timeout_t t);
void  k_heap_free(struct k_heap *h, void *mem);
// stdlib-compatible: malloc(), calloc(), realloc(), free()
```

Пример работы с кучей

```
*** Booting Zephyr OS build v4.3.0 ***
System heap sample

allocated 0,   free 196, max allocated 0,   heap size 256
allocated 156, free 36,  max allocated 156, heap size 256
allocated 100, free 92,  max allocated 156, heap size 256
allocated 0,   free 196, max allocated 156, heap size 256

1 static heap(s) allocated:
  0 - addr 0x10000034  alloc 12, free 180, max alloc 12, size 256
3 heap(s) allocated (including static):
  0 - addr 0x10000034  alloc 12, free 180, max alloc 12, size 256
  1 - addr 0x1000036c  alloc 0,  free 60676, max alloc 0, size 256
  2 - addr 0x10000250  alloc 0,  free 196, max alloc 156, size 256
```

Итоги

Zephyr RTOS предоставляет

- Вытесняющий приоритетный планировщик
- Изолированные потоки с сохранением контекста
- Примитивы синхронизации: семафоры, мьютексы, очереди
- Гибкое управление памятью: статика + динамика
- Модульное ядро (от 2 КБ)
- Поддержку 10+ архитектур, включая RISC-V
- Стек протоколов: TCP/IP, Bluetooth, USB